

МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
им. Н.Э. Баумана

Факультет “Информатика и системы управления”
Кафедра “Системы обработки информации и управления”



Дисциплина “Парадигмы и конструкции языков программирования”

Отчет по домашнему заданию

Выполнил:
Студент группы ИУ5-34Б
Кострыкина Е.Г.
Преподаватель:
Нардид А.Н.

Москва 2025

Цель

Изучение ранее неизвестного языка программирования.

Задание

1. Выберите язык программирования (который Вы ранее не изучали) и (1) напишите по нему реферат с примерами кода или (2) реализуйте на нем небольшой проект (с детальным текстовым описанием).
2. Реферат (проект) может быть посвящен отдельному аспекту (аспектам) языка или содержать решение какой-либо задачи на этом языке.
3. Необходимо установить на свой компьютер компилятор (интерпретатор, транспилятор) этого языка и произвольную среду разработки.
4. В случае написания реферата необходимо разработать и откомпилировать примеры кода (или модифицировать стандартные примеры).
5. В случае создания проекта необходимо детально комментировать код.
6. При написании реферата (создании проекта) необходимо изучить и корректно использовать особенности парадигмы языка и основных конструкций данного языка.
7. Приветствуется написание черновика статьи по результатам выполнения ДЗ. Черновик статьи может быть подготовлен группой студентов, которые исследовали один и тот же аспект в нескольких языках или решили одинаковую задачу на нескольких языках.

Описание проекта

Проект "Космическая оборона" представляет собой классическую 2D-аркадную игру, разработанную на языке программирования Python с использованием библиотеки Pygame. Основная цель проекта - создание полноценной игровой системы с реализацией основных игровых механик, интуитивно понятным интерфейсом и системой сохранения прогресса. Игра относится к жанру шутер, где игрок управляет космическим кораблём и должен уничтожать вражеские объекты, уклоняясь от столкновений с ними. Основной целью проекта являлось создание образовательной игровой платформы, демонстрирующей принципы объектно-ориентированного программирования и работу с графической библиотекой Pygame. В процессе разработки решались следующие задачи:

- Разработка игрового движка с поддержкой графики, анимации и звука
- Создание системы управления игровыми объектами через классы и наследование
- Реализация физики движения и системы коллизий
- Разработка интерфейса пользователя с несколькими экранами
- Создание системы сохранения и загрузки игрового прогресса
- Оптимизация производительности для плавного игрового процесса
- Балансировка игровых механик для обеспечения адекватного уровня сложности

Основные механики

Игровой процесс построен вокруг управления космическим кораблём, который может перемещаться горизонтально по нижней части экрана. Игрок должен уничтожать летящие сверху космические объекты, используя пушку корабля. Основная сложность заключается в необходимости одновременно уклоняться от врагов и точно стрелять по ним. Игра завершается при столкновении корабля с любым вражеским объектом или при успешном завершении всех волн врагов на уровне.

Система врагов

В игре реализовано 6 типов вражеских объектов, каждый со своими характеристиками:

- **Метеориты** - базовые враги с низким здоровьем (10 HP) и высокой скоростью
- **Астероиды** - более живучие цели (30 HP) со средней скоростью
- **Кометы** - сбалансированные враги (20 HP) со средней скоростью
- **Спутники** - особые цели, при попадании по которым отнимаются очки (-60)
- **Метеориты-боссы** - мощные враги (100 HP) с медленным движением
- **Астероиды-боссы** - самые сильные враги (150 HP)

Каждый тип врага имеет уникальный внешний вид и анимацию уничтожения.

Система очков и прогресса

Игра включает сложную систему подсчёта очков:

- За уничтожение обычных врагов начисляются очки, равные их здоровью
- Уничтожение спутника приводит к потере очков
- Сохраняются как общий счёт, так и рекорды для каждого из 5 уровней
- Данные сохраняются в текстовом файле score.txt и загружаются при запуске игры

Архитектура программы

Проект построен по модульной архитектуре с чётким разделением ответственности:

1. **Основной модуль** - инициализация игры, управление глобальными переменными
2. **Классы игровых объектов** - реализация через наследование от базового класса Sprite
3. **Система экранов** - отдельные функции для каждого игрового состояния
4. **Менеджер ресурсов** - загрузка и управление графикой и звуком

Ключевые классы

Класс Spaceship - управляет космическим кораблём игрока. Реализует анимацию через спрайт-лист, обработку столкновений и движение с ограничением границ экрана.

Базовый класс Enemy - содержит общую логику для всех врагов: здоровье, очки, анимацию уничтожения, движение и обработку попаданий. От этого класса наследуются все специализированные типы врагов.

Класс Shot - реализует выстрелы игрока с анимацией и проверкой выхода за границы экрана.

Система отображения

Игра использует три основных экрана:

1. **Стартовый экран** - содержит название игры и кнопки перехода к правилам или выбору уровня
2. **Экран выбора уровня** - показывает доступные уровни в виде сетки 3×3
3. **Игровые экраны** - 5 отдельных уровней с разными наборами врагов

Каждый экран имеет свою фоновую картинку и элементы интерфейса, отрисованные с помощью примитивов Pygame.

Система коллизий

Для точного определения столкновений используется система масок (mask collision). Каждый спрайт создаёт маску на основе своей альфа-канальной текстуры, что позволяет определять столкновения по фактической форме объекта, а не по его ограничивающему прямоугольнику.

Анимационная система

Все движущиеся объекты в игре используют анимацию:

- Корабль игрока имеет 12 кадров анимации (3×4 спрайт-лист)
- Выстрелы анимированы через 5 кадров

- Враги при уничтожении проигрывают одну из 6 случайных анимаций взрыва

Анимация реализована через смену кадров с фиксированной частотой, контролируемой счётчиком обновлений.

Управление временем и событиями

Игра использует два типа событий:

- Стандартные события Pygame (нажатия клавиш, движение мыши)
- Пользовательские события для появления врагов по таймеру

Частота кадров фиксирована на уровне 30 FPS, что обеспечивает стабильную работу на различных системах.

Работа с файловой системой

Проект демонстрирует несколько подходов к работе с файлами:

- Чтение и запись текстовых файлов для сохранения счёта
- Организация ресурсов в структурированные каталоги
- Проверка существования файлов перед загрузкой

Управление памятью

Для предотвращения утечек памяти реализована корректная очистка ресурсов:

- Удаление спрайтов из групп при их уничтожении
- Остановка и выгрузка музыки при смене экранов
- Очистка списков объектов при завершении уровней

Звуковая система

Игра включает полноценную звуковую систему:

- Фоновая музыка для меню и игровых уровней
- Звуковые эффекты для выстрелов и попаданий
- Корректное переключение между звуковыми дорожками

Код программы

main.py

```
import pygame
import os
import sys
import random
import pygame.mixer

pygame.init()

# Константы игры
FPS = 30 # Частота кадров
WIDTH = 500 # Ширина окна
HEIGHT = 600 # Высота окна
sc = 0 # Текущий счёт уровня
SCORE1 = 0 # Рекорд уровня 1
SCORE2 = 0 # Рекорд уровня 2
SCORE3 = 0 # Рекорд уровня 3
SCORE4 = 0 # Рекорд уровня 4
SCORE5 = 0 # Рекорд уровня 5
SHOTS = [] # Список активных выстрелов

# Открытие данных и их считывание из файла score.txt
with open('score.txt', 'a') as f1: # Открытие для добавления, чтобы создать
    файл если его нет
    with open('score.txt', 'r') as f2: # Открытие для чтения
        data = f2.readlines()
        if not data: # Если файл пустой
            SCORE = 0 # Общий счёт
            SCORE1 = 0
            SCORE2 = 0
            SCORE3 = 0
            SCORE4 = 0
            SCORE5 = 0
            f1.write('0\n0\n0\n0\n0\n0')

        else: # Если файл не пустой
            SCORE = int(data[0])
            SCORE1 = int(data[1])
            SCORE2 = int(data[2])
            SCORE3 = int(data[3])
            SCORE4 = int(data[4])
            SCORE5 = int(data[5])

# Создание игрового окна
screen = pygame.display.set_mode((WIDTH, HEIGHT))
clock = pygame.time.Clock()

# Группы спрайтов
all_sprites = pygame.sprite.Group()
enemies = pygame.sprite.Group()

def load_image(name, colorkey=None):
    """Функция для загрузки изображений"""
    fullname = os.path.join('data', name)
    if not os.path.isfile(fullname): # Проверка существования файла
        print(f"Файл с изображением '{fullname}' не найден")
        sys.exit()
    image = pygame.image.load(fullname)
```

```

if colorkey is not None: # Если указан цвет для прозрачности
    image = image.convert()
    if colorkey == -1:
        colorkey = image.get_at((0, 0))
    image.set_colorkey(colorkey)
else:
    image = image.convert_alpha()
return image

def terminate():
    """Функция для отключения pygame"""
    pygame.quit() # Завершение работы Pygame
    sys.exit() # Выход из программы

def load_level(en, k):
    """Функция для появления врагов на уровне"""
    i = en[k] # Получение данных о враге
    if i[0] == 'Meteorite': # Создание метеорита
        m = Meteorite()
        m.rect.x = i[1] # Установка координаты X
        m.rect.y = -100 # Начальная позиция выше экрана
    if i[0] == 'Asteroid': # Создание астероида
        m = Asteroid()
        m.rect.x = i[1]
        m.rect.y = -100
    if i[0] == 'Comet': # Создание кометы
        m = Comet()
        m.rect.x = i[1]
        m.rect.y = -100
    if i[0] == 'AsteroidBoss': # Создание босса-астероида
        m = AsteroidBoss()
        m.rect.x = i[1]
        m.rect.y = -100
    if i[0] == 'MeteoriteBoss': # Создание босса-метеорита
        m = MeteoriteBoss()
        m.rect.x = i[1]
        m.rect.y = -100
    if i[0] == 'Satellite': # Создание спутника
        m = Satellite()
        m.rect.x = i[1]
        m.rect.y = -100

# Пример работы функций экранов прокомментирован
# в функции level_1()

def start_screen():
    """Функция для отображения и работы стартового окна"""
    fon = pygame.transform.scale(load_image(f'Picture/SpaceBackground1.png'),
    (WIDTH, HEIGHT))
    screen.blit(fon, (0, 0))

    main_text = "КОСМИЧЕСКАЯ ОБОРОНА"
    font = pygame.font.SysFont('MathSansBold', 35)
    text = font.render(main_text, True, pygame.Color(252, 242, 255))
    m_rect = text.get_rect()
    m_rect.top = 210
    m_rect.x = WIDTH // 2 - text.get_width() // 2
    pygame.draw.rect(screen, (97, 69, 107), (m_rect.x - 10, m_rect.y - 10,
    m_rect.width + 20, m_rect.height
    + 20))
    screen.blit(text, m_rect)

```

```

text_coord = 345
x = 0
h = 0
w = 0
text_list = ["Правила игры", "Играть"]
font = pygame.font.SysFont('MathSans', 27)
for line in text_list:
    text = font.render(line, True, pygame.Color(97, 69, 107))
    m_rect = text.get_rect()
    m_rect.top = text_coord
    m_rect.x = WIDTH // 2 - text.get_width() // 2
    if h == 0:
        h = m_rect.h + 20
        x = m_rect.x - 10
        w = m_rect.width + 20
    pygame.draw.rect(screen, (252, 242, 255), (x, m_rect.y - 10,
                                                    w, h))

    screen.blit(text, m_rect)
    text_coord -= 45

while True:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            terminate()
        elif event.type == pygame.MOUSEBUTTONDOWN:
            if x <= event.pos[0] <= x + w:
                if m_rect.y - 10 <= event.pos[1] <= m_rect.y - 10 + h:
                    level_screen()
                if m_rect.y + 35 <= event.pos[1] <= m_rect.y + 35 + h:
                    rules_screen()
    pygame.display.flip()
    clock.tick(FPS)

def level_screen():
    """Функция для отображения и работы окна с уровнями"""
    fon = pygame.transform.scale(load_image(f'Picture/SpaceBackground2.png'),
    (WIDTH, HEIGHT))
    screen.blit(fon, (0, 0))
    pygame.draw.rect(screen, (252, 242, 255), (10, 10,
                                                    40, 40))
    pygame.draw.polygon(screen, (97, 69, 107), [(20, 30), (35, 40), (35,
20)], 3)
    font = pygame.font.SysFont('MathSansBold', 30)
    text = font.render(f'СЧЁТ: {SCORE}', True, pygame.Color(252, 242, 255))
    screen.blit(text, (WIDTH // 2 - text.get_width() // 2, 25))

    text_list = ["Уровень 1", "Уровень 2", "Уровень 3", "Уровень 4",
                  "Уровень 5", "Скоро...", "Скоро...", "Скоро...", "Скоро..."]
    font = pygame.font.SysFont('MathSansBold', 27)
    text_y = 140
    y = 100
    k = 0
    for i in range(3):
        x = 60
        text_x = 73
        for j in range(3):
            text = font.render(text_list[k], True, pygame.Color(252, 242,
255))

            m_rect = text.get_rect()
            m_rect.y = text_y
            m_rect.x = text_x
            pygame.draw.rect(screen, (97, 69, 107), (x, y, 120, 100))

```



```

        screen.blit(text, m_rect)
        text_x += 140
        x += 140
        k += 1
        text_y += 140
        y += 140

while True:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            terminate()
        elif event.type == pygame.MOUSEBUTTONDOWN:
            if 10 <= event.pos[0] <= 50 and 10 <= event.pos[1] <= 50:
                start_screen()
            elif 60 <= event.pos[0] <= 180:
                if 80 <= event.pos[1] <= 180:
                    level_1()
                elif 220 <= event.pos[1] <= 320:
                    level_4()
            elif 200 <= event.pos[0] <= 320:
                if 80 <= event.pos[1] <= 180:
                    level_2()
                elif 220 <= event.pos[1] <= 320:
                    level_5()
            elif 340 <= event.pos[0] <= 460 and 80 <= event.pos[1] <=
180:
                level_3()
    pygame.display.flip()
    clock.tick(FPS)

def rules_screen():
    """Функция отрисовки и работы окна с правилами"""
    fon = pygame.transform.scale(load_image(f'Picture/SpaceBackground3.png'),
(WIDTH, HEIGHT))
    screen.blit(fon, (0, 0))
    pygame.draw.rect(screen, (252, 242, 255), (10, 10,
40, 40))
    pygame.draw.polygon(screen, (97, 69, 107), [(20, 30), (35, 40), (35,
20)], 3)

    pygame.draw.rect(screen, (97, 69, 107), (50, 100,
400, 410))

    text_coord = 115
    text_list = ["Вам предстоит расчистить путь",
" для своего космического корабля от",
" различных препятствий: комет,",
" астероидов, метеоритов.",
"У каждой цели разная степень",
"живучести, скорость и очки",
"за её уничтожение",
"Если вы попадёте по спутнику,",
"то счёт понизится.",
"Не забудьте уворачиваться от целей.",
"Стрелочки ВПРАВО и ВЛЕВО - передвижение",
" космического корабля;",
"ПРОБЕЛ - использование пушки."]
    font = pygame.font.SysFont('MathSans', 23)
    for line in text_list:
        text = font.render(line, True, pygame.Color(252, 242, 255))
        m_rect = text.get_rect()
        m_rect.top = text_coord
        m_rect.x = 75

```

```

        screen.blit(text, m_rect)
        text_coord += 30

while True:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            terminate()
        elif event.type == pygame.MOUSEBUTTONDOWN:
            if 10 <= event.pos[0] <= 50 and 10 <= event.pos[1] <= 50:
                start_screen()
    pygame.display.flip()
    clock.tick(FPS)

def level_1():
    """Функция отрисовки и работы 1 уровня"""
    # Загрузка фона
    fon = pygame.transform.scale(load_image(f'Picture/SpaceBackground3.png'),
                                  (WIDTH, HEIGHT))

    # Подготовка перед работой с глобальными переменными
    global sc, SHOTS, SCORE, SCORE1
    sc = 0
    SHOTS = []

    # Настройка музыки
    pygame.mixer.music.pause()
    pygame.mixer.music.load(di + 'music.mp3')
    pygame.mixer.music.play(-1)
    spaceship = Spaceship()

    # Список из кортежей вида: (цель на уровне, её координата по иксу)
    en = [('Comet', 100), ('Meteorite', 200), ('Comet', 300),
          ('Asteroid', 75), ('Satellite', 100), ('Comet', 200),
          ('Meteorite', 275), ('Comet', 350), ('Asteroid', 250),
          ('Meteorite', 200), ('Comet', 300), ('Asteroid', 150),
          ('Satellite', 300), ('Comet', 200), ('Meteorite', 275),
          ('Comet', 350), ('Asteroid', 250)]

    # Создание своего события по таймингу
    myeventtype = pygame.USEREVENT + 1
    pygame.time.set_timer(myeventtype, 2000)
    k = 0

while True:
    keys = pygame.key.get_pressed()
    # Движение при нажатии на кнопки.
    # Его мы решили реализовать не в общем
    # с другими event цикле, чтобы корабль
    # мог двигаться с зажатыми кнопкой
    if keys[pygame.K_LEFT] and spaceship.rect.x > 5\
        and not spaceship.fl:
        move(spaceship, 'left')
    if keys[pygame.K_RIGHT] and spaceship.rect.x < 345\
        and not spaceship.fl:
        move(spaceship, 'right')
    for event in pygame.event.get():
        # Обработка других event
        if event.type == pygame.QUIT:
            # Отключение pygame
            terminate()
        elif event.type == pygame.MOUSEBUTTONDOWN:
            if 10 <= event.pos[0] <= 50 and 10 <= event.pos[1] <= 50\
                and not spaceship.fl:
                # Нажатие кнопки перехода в меню уровней

```

```

        spaceship.kill()
        pygame.mixer.music.stop()
        pygame.mixer.music.load(di + 'menu_music.mp3')
        pygame.mixer.music.play(-1)
        for i in enemies:
            i.kill()
        for i in SHOTS:
            i.kill()
        level_screen()
    if 175 <= event.pos[0] <= 325 and 300 <= event.pos[1] <= 350\
        and spaceship.fl:
        # Нажатие кнопки перехода в меню уровней при завершении
        # уровня
        spaceship.kill()
        pygame.mixer.music.stop()
        pygame.mixer.music.load(di + 'menu_music.mp3')
        pygame.mixer.music.play(-1)
        for i in enemies:
            i.kill()
        for i in SHOTS:
            i.kill()
        level_screen()
    elif event.type == pygame.KEYDOWN and event.key ==
pygame.K_SPACE\
        and not spaceship.fl:
        # Создание объектов-выстрелов при нажатие пробела
        SHOTS.append(Shot(spaceship.rect.x, spaceship.rect.y))
        s.play()
    elif event.type == myeventtype and k < len(en)\
        and not spaceship.fl:
        # Обработка event по таймингу
        load_level(en, k)
        k += 1

# Создание интерфейса
screen.blit(fon, (0, 0))
pygame.draw.rect(screen, (252, 242, 255), (10, 10,
40, 40))
pygame.draw.rect(screen, (252, 242, 255), (430, 10,
60, 40))
font = pygame.font.SysFont('MathSansBold', 25)
text = font.render(f'{sc}', True, pygame.Color(97, 69, 107))
screen.blit(text, (460 - text.get_width() // 2, 23))
pygame.draw.polygon(screen, (97, 69, 107), [(20, 30), (35, 40), (35,
20)], 3)

enemies.draw(screen)
all_sprites.draw(screen)
if not spaceship.fl:
    # Обновление состояния целей и корабля
    # с предварительной проверкой отсутствия
    # касания целей и корабля
    enemies.update()
    all_sprites.update()
else:
    # При касании создаётся интерфейс "Поражение"
    pygame.draw.rect(screen, (97, 69, 107), (100, 200,
300, 200))
    pygame.draw.rect(screen, (252, 242, 255), (175, 300,
150, 50))
    font = pygame.font.SysFont('MathSansBold', 30)
    text = font.render('ПОРАЖЕНИЕ', True, pygame.Color(252, 242,
255))
    screen.blit(text, (185, 230))

```

```

font = pygame.font.SysFont('MathSansBold', 27)
text = font.render('Вернуться', True, pygame.Color(97, 69, 107))
screen.blit(text, (205, 315))
if not enemies and k == len(en):
    # В той ситуации, когда спрайтов на уровне уже нет,
    # и все спрайты из списка en инициализированы
    # создаётся интерфейс "Победа" и сохраняются
    # счёт уровня в score.txt
    SCORE += sc - SCORE1
    SCORE1 = sc
    with open('score.txt', 'w') as f:
        f.write(f'{str(SCORE)}\n{str(SCORE1)}\n{str(SCORE2)}\n'
                f'\n{str(SCORE3)}\n{str(SCORE4)}\n{str(SCORE5)}\n')
    spaceship.fl = 1
    pygame.draw.rect(screen, (97, 69, 107), (100, 200,
                                                300, 200))
    pygame.draw.rect(screen, (252, 242, 255), (175, 300,
                                                150, 50))
    font = pygame.font.SysFont('MathSansBold', 30)
    text = font.render('ПОБЕДА', True, pygame.Color(252, 242, 255))
    screen.blit(text, (205, 230))
    font = pygame.font.SysFont('MathSansBold', 27)
    text = font.render('Вернуться', True, pygame.Color(97, 69, 107))
    screen.blit(text, (205, 315))
pygame.display.flip()
clock.tick(FPS)

```

```

def level_2():
    """Функция отрисовки и работы 2 уровня"""
    fon = pygame.transform.scale(load_image(f'Picture/SpaceBackground3.png'),
                                  (WIDTH, HEIGHT))

    global sc, SHOTS, SCORE, SCORE2
    sc = 0

    pygame.mixer.music.pause()
    pygame.mixer.music.load(di + 'music.mp3')
    pygame.mixer.music.play(-1)
    spaceship = Spaceship()

    SHOTS = []
    en = [('Asteroid', 200), ('Comet', 100), ('Meteorite', 150),
          ('Comet', 300), ('Meteorite', 250), ('Satellite', 300),
          ('Comet', 280), ('Asteroid', 50), ('Satellite', 100),
          ('Meteorite', 250), ('Comet', 175), ('Asteroid', 200),
          ('Asteroid', 250), ('Meteorite', 200), ('Comet', 220),
          ('Asteroid', 200), ('Comet', 100), ('Meteorite', 150),
          ('Comet', 300), ('Meteorite', 250), ('Satellite', 300),
          ('Comet', 350), ('Asteroid', 50), ('Meteorite', 250),
          ('Comet', 175), ('Meteorite', 200), ('Comet', 150)]
    myeventtype = pygame.USEREVENT + 1
    pygame.time.set_timer(myeventtype, 2000)
    k = 0

    while True:
        keys = pygame.key.get_pressed()
        if keys[pygame.K_LEFT] and spaceship.rect.x > 5\
            and not spaceship.fl:
            move(spaceship, 'left')
        if keys[pygame.K_RIGHT] and spaceship.rect.x < 345\
            and not spaceship.fl:
            move(spaceship, 'right')
        for event in pygame.event.get():
            if event.type == pygame.QUIT:

```

```

        terminate()
    elif event.type == pygame.MOUSEBUTTONDOWN:
        if 10 <= event.pos[0] <= 50 and 10 <= event.pos[1] <= 50\
            and not spaceship.fl:
            spaceship.kill()
            pygame.mixer.music.stop()
            pygame.mixer.music.load(di + 'menu_music.mp3')
            pygame.mixer.music.play(-1)
            for i in enemies:
                i.kill()
            for i in SHOTS:
                i.kill()
            level_screen()
        if 175 <= event.pos[0] <= 325 and 300 <= event.pos[1] <= 350\
            and spaceship.fl:
            spaceship.kill()
            pygame.mixer.music.stop()
            pygame.mixer.music.load(di + 'menu_music.mp3')
            pygame.mixer.music.play(-1)
            for i in enemies:
                i.kill()
            for i in SHOTS:
                i.kill()
            level_screen()
    elif event.type == pygame.KEYDOWN and event.key ==
pygame.K_SPACE\
        and not spaceship.fl:
        SHOTS.append(Shot(spaceship.rect.x, spaceship.rect.y))
        s.play()
    elif event.type == myeventtype and k < len(en)\
        and not spaceship.fl:
        load_level(en, k)
        k += 1

screen.blit(fon, (0, 0))
pygame.draw.rect(screen, (252, 242, 255), (10, 10,
40, 40))
pygame.draw.rect(screen, (252, 242, 255), (430, 10,
60, 40))
font = pygame.font.SysFont('MathSansBold', 25)
text = font.render(f'{sc}', True, pygame.Color(97, 69, 107))
screen.blit(text, (460 - text.get_width() // 2, 23))
pygame.draw.polygon(screen, (97, 69, 107), [(20, 30), (35, 40), (35,
20)], 3)

enemies.draw(screen)
all_sprites.draw(screen)
if not spaceship.fl:
    enemies.update()
    all_sprites.update()
else:
    pygame.draw.rect(screen, (97, 69, 107), (100, 200,
300, 200))
    pygame.draw.rect(screen, (252, 242, 255), (175, 300,
150, 50))
    font = pygame.font.SysFont('MathSansBold', 30)
    text = font.render('ПОПАЖЕНИЕ', True, pygame.Color(252, 242,
255))
    screen.blit(text, (185, 230))
    font = pygame.font.SysFont('MathSansBold', 27)
    text = font.render('Вернуться', True, pygame.Color(97, 69, 107))
    screen.blit(text, (205, 315))
if not enemies and k == len(en):
    SCORE += sc - SCORE2

```

```

SCORE2 = sc
with open('score.txt', 'w') as f:
    f.write(f'{str(SCORE)}\n{str(SCORE1)}\n{str(SCORE2)}\n{str(SCORE3)}\n{str(SCORE4)}\n{str(SCORE5)}\n')
spaceship.fl = 1
pygame.draw.rect(screen, (97, 69, 107), (100, 200, 300, 200))
pygame.draw.rect(screen, (252, 242, 255), (175, 300, 150, 50))

font = pygame.font.SysFont('MathSansBold', 30)
text = font.render('ПОБЕДА', True, pygame.Color(252, 242, 255))
screen.blit(text, (205, 230))
font = pygame.font.SysFont('MathSansBold', 27)
text = font.render('Вернуться', True, pygame.Color(97, 69, 107))
screen.blit(text, (205, 315))
pygame.display.flip()
clock.tick(FPS)

def level_3():
    """Функция отрисовки и работы 3 уровня"""
    fon = pygame.transform.scale(load_image(f'Picture/SpaceBackground3.png'), (WIDTH, HEIGHT))

    global sc, SHOTS, SCORE, SCORE3
    sc = 0

    pygame.mixer.music.stop()
    pygame.mixer.music.load(di + 'music.mp3')
    pygame.mixer.music.play(-1)
    spaceship = Spaceship()

    SHOTS = []
    en = [('Meteorite', 200), ('Comet', 100), ('Comet', 300), ('Meteorite', 200), ('Asteroid', 300), ('Asteroid', 250), ('Asteroid', 200), ('Asteroid', 150), ('Asteroid', 100), ('Satellite', 150), ('Satellite', 350), ('Comet', 200), ('Meteorite', 300), ('Asteroid', 100), ('Comet', 80), ('', 0), ('Asteroid', 200), ('Meteorite', 100), ('Comet', 150), ('Comet', 300), ('Meteorite', 200), ('', 0), ('Satellite', 150), ('Satellite', 350), ('', 0), ('MeteoriteBoss', 100)]

    myeventtype = pygame.USEREVENT + 1
    pygame.time.set_timer(myeventtype, 2000)
    k = 0

    while True:
        keys = pygame.key.get_pressed()
        if keys[pygame.K_LEFT] and spaceship.rect.x > 5\
            and not spaceship.fl:
            move(spaceship, 'left')
        if keys[pygame.K_RIGHT] and spaceship.rect.x < 345\
            and not spaceship.fl:
            move(spaceship, 'right')
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                terminate()
            elif event.type == pygame.MOUSEBUTTONDOWN:
                if 10 <= event.pos[0] <= 50 and 10 <= event.pos[1] <= 50 \
                    and not spaceship.fl:
                    spaceship.kill()
                    pygame.mixer.music.stop()
                    pygame.mixer.music.load(di + 'menu_music.mp3')
                    pygame.mixer.music.play(-1)
                    for i in enemies:

```

```

        i.kill()
    for i in SHOTS:
        i.kill()
    level_screen()
    if 175 <= event.pos[0] <= 325 and 300 <= event.pos[1] <= 350\
        and spaceship.fl:
        spaceship.kill()
        pygame.mixer.music.stop()
        pygame.mixer.music.load(di + 'menu_music.mp3')
        pygame.mixer.music.play(-1)
        for i in enemies:
            i.kill()
        for i in SHOTS:
            i.kill()
        level_screen()
    elif event.type == pygame.KEYDOWN and event.key ==
pygame.K_SPACE\
        and not spaceship.fl:
        SHOTS.append(Shot(spaceship.rect.x, spaceship.rect.y))
        s.play()
    elif event.type == myeventtype and k < len(en)\
        and not spaceship.fl:
        load_level(en, k)
        k += 1

screen.blit(fon, (0, 0))
pygame.draw.rect(screen, (252, 242, 255), (10, 10,
40, 40))
pygame.draw.rect(screen, (252, 242, 255), (430, 10,
60, 40))
font = pygame.font.SysFont('MathSansBold', 25)
text = font.render(f'{sc}', True, pygame.Color(97, 69, 107))
screen.blit(text, (460 - text.get_width() // 2, 23))
pygame.draw.polygon(screen, (97, 69, 107), [(20, 30), (35, 40), (35,
20)], 3)

enemies.draw(screen)
all_sprites.draw(screen)
if not spaceship.fl:
    enemies.update()
    all_sprites.update()
else:
    pygame.draw.rect(screen, (97, 69, 107), (100, 200,
300, 200))
    pygame.draw.rect(screen, (252, 242, 255), (175, 300,
150, 50))
    font = pygame.font.SysFont('MathSansBold', 30)
    text = font.render('ПОПАЖЕНИЕ', True, pygame.Color(252, 242,
255))
    screen.blit(text, (185, 230))
    font = pygame.font.SysFont('MathSansBold', 27)
    text = font.render('Вернуться', True, pygame.Color(97, 69, 107))
    screen.blit(text, (205, 315))
if not enemies and k == len(en):
    SCORE += sc - SCORE3
    SCORE3 = sc
    with open('score.txt', 'w') as f:
        f.write(f'{str(SCORE)}\n{str(SCORE1)}\n{str(SCORE2)}\n'
            f'\n{str(SCORE3)}\n{str(SCORE4)}\n{str(SCORE5)}\n')
    spaceship.fl = 1
    pygame.draw.rect(screen, (97, 69, 107), (100, 200,
300, 200))
    pygame.draw.rect(screen, (252, 242, 255), (175, 300,
150, 50))

```

```

        font = pygame.font.SysFont('MathSansBold', 30)
        text = font.render('ПОБЕДА', True, pygame.Color(252, 242, 255))
        screen.blit(text, (205, 230))
        font = pygame.font.SysFont('MathSansBold', 27)
        text = font.render('Вернуться', True, pygame.Color(97, 69, 107))
        screen.blit(text, (205, 315))
    pygame.display.flip()
    clock.tick(FPS)

def level_4():
    """Функция отрисовки и работы 4 уровня"""
    fon = pygame.transform.scale(load_image(f'Picture/SpaceBackground3.png'),
                                   (WIDTH, HEIGHT))

    global sc, SHOTS, SCORE, SCORE4
    sc = 0

    pygame.mixer.music.stop()
    pygame.mixer.music.load(di + 'music.mp3')
    pygame.mixer.music.play(-1)
    spaceship = Spaceship()

    SHOTS = []
    en = [('Comet', 100), ('Meteorite', 200), ('Comet', 300),
          ('Asteroid', 75), ('Satellite', 100), ('Comet', 200),
          ('Meteorite', 275), ('Comet', 350), ('Asteroid', 250),
          ('Meteorite', 200), ('Comet', 300), ('Asteroid', 150),
          ('Satellite', 300), ('Asteroid', 250), ('Meteorite', 200),
          ('Comet', 220), ('Asteroid', 200), ('Comet', 100),
          ('Meteorite', 150), ('Comet', 300), ('Meteorite', 250),
          ('Satellite', 300), ('Comet', 350), ('Asteroid', 50),
          ('Meteorite', 250), ('Comet', 175), ('Meteorite', 200),
          ('Comet', 150), ('Comet', 100), ('Meteorite', 200), ('Comet', 300),
          ('Asteroid', 75), ('Satellite', 100), ('Comet', 200),
          ('Meteorite', 275), ('Comet', 350), ('Asteroid', 250),
          ('Meteorite', 200), ('Comet', 300), ('Asteroid', 150),
          ('Satellite', 300), ('Asteroid', 250)]

    myeventtype = pygame.USEREVENT + 1
    pygame.time.set_timer(myeventtype, 2000)
    k = 0

    while True:
        keys = pygame.key.get_pressed()
        if keys[pygame.K_LEFT] and spaceship.rect.x > 5\
            and not spaceship.fl:
            move(spaceship, 'left')
        if keys[pygame.K_RIGHT] and spaceship.rect.x < 345\
            and not spaceship.fl:
            move(spaceship, 'right')
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                terminate()
            elif event.type == pygame.MOUSEBUTTONDOWN:
                if 10 <= event.pos[0] <= 50 and 10 <= event.pos[1] <= 50\
                    and not spaceship.fl:
                    spaceship.kill()
                    pygame.mixer.music.stop()
                    pygame.mixer.music.load(di + 'menu_music.mp3')
                    pygame.mixer.music.play(-1)
                    for i in enemies:
                        i.kill()
                    for i in SHOTS:
                        i.kill()
                    level_screen()

```



```

        if 175 <= event.pos[0] <= 325 and 300 <= event.pos[1] <= 350\
            and spaceship.fl:
                spaceship.kill()
                pygame.mixer.music.stop()
                pygame.mixer.music.load(di + 'menu_music.mp3')
                pygame.mixer.music.play(-1)
                for i in enemies:
                    i.kill()
                for i in SHOTS:
                    i.kill()
                level_screen()
            elif event.type == pygame.KEYDOWN and event.key ==
pygame.K_SPACE\
                and not spaceship.fl:
                    SHOTS.append(Shot(spaceship.rect.x, spaceship.rect.y))
                    s.play()
            elif event.type == myeventtype and k < len(en)\
                and not spaceship.fl:
                    load_level(en, k)
                    k += 1

screen.blit(fon, (0, 0))
pygame.draw.rect(screen, (252, 242, 255), (10, 10,
                                                40, 40))
pygame.draw.rect(screen, (252, 242, 255), (430, 10,
                                                60, 40))
font = pygame.font.SysFont('MathSansBold', 25)
text = font.render(f'{sc}', True, pygame.Color(97, 69, 107))
screen.blit(text, (460 - text.get_width() // 2, 23))
pygame.draw.polygon(screen, (97, 69, 107), [(20, 30), (35, 40), (35,
20)], 3)

enemies.draw(screen)
all_sprites.draw(screen)
if not spaceship.fl:
    enemies.update()
    all_sprites.update()
else:
    pygame.draw.rect(screen, (97, 69, 107), (100, 200,
                                                300, 200))
    pygame.draw.rect(screen, (252, 242, 255), (175, 300,
                                                150, 50))
    font = pygame.font.SysFont('MathSansBold', 30)
    text = font.render('ПОПАЖЕНИЕ', True, pygame.Color(252, 242,
255))
    screen.blit(text, (185, 230))
    font = pygame.font.SysFont('MathSansBold', 27)
    text = font.render('Вернуться', True, pygame.Color(97, 69, 107))
    screen.blit(text, (205, 315))
if not enemies and k == len(en):
    SCORE += sc - SCORE4
    SCORE4 = sc
    with open('score.txt', 'w') as f:
        f.write(f'{str(SCORE)}\n{str(SCORE1)}\n{str(SCORE2)}\n'
                f'\n{str(SCORE3)}\n{str(SCORE4)}\n{str(SCORE5)}\n')
    spaceship.fl = 1
    pygame.draw.rect(screen, (97, 69, 107), (100, 200,
                                                300, 200))
    pygame.draw.rect(screen, (252, 242, 255), (175, 300,
                                                150, 50))
    font = pygame.font.SysFont('MathSansBold', 30)
    text = font.render('ПОБЕДА', True, pygame.Color(252, 242, 255))
    screen.blit(text, (205, 230))
    font = pygame.font.SysFont('MathSansBold', 27)

```

```

        text = font.render('Вернуться', True, pygame.Color(97, 69, 107))
        screen.blit(text, (205, 315))
    pygame.display.flip()
    clock.tick(FPS)

def level_5():
    """Функция отрисовки и работы 5 уровня"""
    fon = pygame.transform.scale(load_image(f'Picture/SpaceBackground3.png'),
                                   (WIDTH, HEIGHT))

    global sc, SHOTS, SCORE, SCORE5
    sc = 0

    pygame.mixer.music.stop()
    pygame.mixer.music.load(di + 'music.mp3')
    pygame.mixer.music.play(-1)
    spaceship = Spaceship()

    SHOTS = []
    en = [('Comet', 100), ('Meteorite', 200), ('Comet', 300),
          ('Asteroid', 75), ('Satellite', 100), ('Comet', 200),
          ('Meteorite', 275), ('Comet', 350), ('Asteroid', 250),
          ('Meteorite', 200), ('Comet', 300), ('Asteroid', 150),
          ('Satellite', 300), ('Asteroid', 250), ('Meteorite', 200),
          ('Comet', 220), ('Asteroid', 200), ('Comet', 100),
          ('Meteorite', 150), ('Comet', 300), ('Meteorite', 250),
          ('Satellite', 300), ('Comet', 350), ('Asteroid', 50),
          ('Comet', 175), ('Meteorite', 200),
          ('Asteroid', 100), ('Comet', 80),
          ('', 0), ('Asteroid', 200), ('Meteorite', 100),
          ('Comet', 150), ('Comet', 300), ('Meteorite', 200),
          ('', 0), ('Satellite', 150),
          ('Satellite', 350), ('', 0), ('MeteoriteBoss', 100),
          ('Meteorite', 275), ('Comet', 350), ('Asteroid', 250),
          ('Meteorite', 200), ('Comet', 300), ('Asteroid', 150),
          ('Satellite', 300), ('Asteroid', 250), ('Satellite', 75),
          ('AsteroidBoss', 200)]

    myeventtype = pygame.USEREVENT + 1
    pygame.time.set_timer(myeventtype, 2000)
    k = 0

    while True:
        keys = pygame.key.get_pressed()
        if keys[pygame.K_LEFT] and spaceship.rect.x > 5\
            and not spaceship.fl:
            move(spaceship, 'left')
        if keys[pygame.K_RIGHT] and spaceship.rect.x < 345\
            and not spaceship.fl:
            move(spaceship, 'right')
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                terminate()
            elif event.type == pygame.MOUSEBUTTONDOWN:
                if 10 <= event.pos[0] <= 50 and 10 <= event.pos[1] <= 50\
                    and not spaceship.fl:
                    spaceship.kill()
                    pygame.mixer.music.stop()
                    pygame.mixer.music.load(di + 'menu_music.mp3')
                    pygame.mixer.music.play(-1)
                    for i in enemies:
                        i.kill()
                    for i in SHOTS:
                        i.kill()
                    level_screen()

```

```

        if 175 <= event.pos[0] <= 325 and 300 <= event.pos[1] <= 350\
            and spaceship.fl:
                spaceship.kill()
                pygame.mixer.music.stop()
                pygame.mixer.music.load(di + 'menu_music.mp3')
                pygame.mixer.music.play(-1)
                for i in enemies:
                    i.kill()
                for i in SHOTS:
                    i.kill()
                level_screen()
            elif event.type == pygame.KEYDOWN and event.key ==
pygame.K_SPACE\
                and not spaceship.fl:
                    SHOTS.append(Shot(spaceship.rect.x, spaceship.rect.y))
                    s.play()
            elif event.type == myeventtype and k < len(en)\
                and not spaceship.fl:
                    load_level(en, k)
                    k += 1

screen.blit(fon, (0, 0))
pygame.draw.rect(screen, (252, 242, 255), (10, 10,
                                                40, 40))
pygame.draw.rect(screen, (252, 242, 255), (430, 10,
                                                60, 40))
font = pygame.font.SysFont('MathSansBold', 25)
text = font.render(f'{sc}', True, pygame.Color(97, 69, 107))
screen.blit(text, (460 - text.get_width() // 2, 23))
pygame.draw.polygon(screen, (97, 69, 107), [(20, 30), (35, 40), (35,
20)], 3)

enemies.draw(screen)
all_sprites.draw(screen)
if not spaceship.fl:
    enemies.update()
    all_sprites.update()
else:
    pygame.draw.rect(screen, (97, 69, 107), (100, 200,
                                                300, 200))
    pygame.draw.rect(screen, (252, 242, 255), (175, 300,
                                                150, 50))
    font = pygame.font.SysFont('MathSansBold', 30)
    text = font.render('ПОПАЖЕНИЕ', True, pygame.Color(252, 242,
255))
    screen.blit(text, (185, 230))
    font = pygame.font.SysFont('MathSansBold', 27)
    text = font.render('Вернуться', True, pygame.Color(97, 69, 107))
    screen.blit(text, (205, 315))
if not enemies and k == len(en):
    SCORE += sc - SCORE5
    SCORE5 = sc
    with open('score.txt', 'w') as f:
        f.write(f'{str(SCORE)}\n{str(SCORE1)}\n{str(SCORE2)}\n'
                f'\n{str(SCORE3)}\n{str(SCORE4)}\n{str(SCORE5)}\n')
    spaceship.fl = 1
    pygame.draw.rect(screen, (97, 69, 107), (100, 200,
                                                300, 200))
    pygame.draw.rect(screen, (252, 242, 255), (175, 300,
                                                150, 50))
    font = pygame.font.SysFont('MathSansBold', 30)
    text = font.render('ПОБЕДА', True, pygame.Color(252, 242, 255))
    screen.blit(text, (205, 230))
    font = pygame.font.SysFont('MathSansBold', 27)

```

```

        text = font.render('Вернуться', True, pygame.Color(97, 69, 107))
        screen.blit(text, (205, 315))
    pygame.display.flip()
    clock.tick(FPS)

class Spaceship(pygame.sprite.Sprite):
    """Класс создания и размещения на окне космического корабля"""
    def __init__(self):
        super().__init__(all_sprites) # Добавление в группу всех спрайтов
        self.frames = [] # Список кадров анимации
        # Загрузка и нарезка спрайт-листа корабля

    self.cut_sheet(pygame.transform.scale(load_image("Picture/spaceship.png"),
                                             (450, 600)), 3, 4)

        self.cur_frame = 0 # Текущий кадр анимации
        self.image = self.frames[self.cur_frame] # Текущее изображение
        self.rect = self.rect.move(175, 440) # Позиция корабля
        self.k = 0 # Счётчик для анимации
        self.speed = 30 # Скорость движения
        self.fl = 0 # Флаг состояния (0-игра, 1-поражение/победа)
        self.mask = pygame.mask.from_surface(self.image) # Маска для
        коллизий

    def cut_sheet(self, sheet, columns, rows):
        """Функция для деления изображения на фреймы и их последующего
        анимирования"""
        self.rect = pygame.Rect(0, 0, sheet.get_width() // columns,
                                sheet.get_height() // rows)

        for j in range(rows):
            for i in range(columns):
                frame_location = (self.rect.w * i, self.rect.h * j)
                self.frames.append(sheet.subsurface(pygame.Rect(
                    frame_location, self.rect.size)))

    def update(self):
        """Функция для обновления состояния корабля"""
        for i in enemies: # Проверка столкновений с врагами
            if pygame.sprite.collide_mask(self, i):
                self.fl = 1

        self.k += 1
        if self.k % 5 == 0: # Каждые 5 кадров меняем кадр анимации
            self.cur_frame = (self.cur_frame + 1) % len(self.frames)
            self.image = self.frames[self.cur_frame]

class Enemy(pygame.sprite.Sprite):
    """Класс для создания всех целей и их расположения"""
    def __init__(self):
        super().__init__(enemies) # Добавление в группу врагов
        self.health = 0 # Здоровье
        self.frames = [] # Кадры анимации уничтожения
        self.score = 0 # Очки за уничтожение
        self.cur_frame = 0 # Текущий кадр
        self.k = 0 # Счётчик для анимации
        # Массив для случайного выбора анимации уничтожения
        hits = ['hit1', 'hit2', 'hit3', 'hit4', 'hit5', 'hit6']
        self.hit = random.choice(hits)
        if self.hit == 'hit1' or self.hit == 'hit3':

    self.cut_sheet(pygame.transform.scale(load_image(f"Picture/{self.hit}.png"),
                                             (64, 320)), 1, 5)

        self.n = 25
    else:

```

```

self.cut_sheet(pygame.transform.scale(load_image(f"Picture/{self.hit}.png"),
                                         (64, 448)), 1, 7)

self.n = 35

def update(self):
    """Функция для обновления состояния целей"""
    global sc
    global SHOTS
    popped_shots = [] # Временный список для выстрелов
    if self.health: # Если враг жив
        for shot in SHOTS: # Проверка столкновений с выстрелами
            if pygame.sprite.collide_mask(self, shot):
                shot.kill()
                self.health -= 10
            else:
                popped_shots.append(shot)
        SHOTS = popped_shots
    if self.health == 0 and self.k == 0: # Если враг только что
уничтожен
        if sc + self.score >= 0: # Обновление счёта
            sc += self.score
        else:
            sc = 0
        # Подготовка к анимации уничтожения
        w = self.image.get_width()
        h = self.image.get_height()
        self.image = self.frames[self.cur_frame]
        self.rect.x += w // 4
        self.rect.y += h // 4
        s2 = pygame.mixer.Sound('data/Sound/hit1.wav') # Звук
уничтожения
        s2.play()
        if self.health == 0 and self.k < self.n: # Анимация уничтожения
            self.k += 1
            if self.k % 5 == 0:
                self.cur_frame = (self.cur_frame + 1) % len(self.frames)
                self.image = self.frames[self.cur_frame]
        if self.k >= self.n:
            self.kill()
        if self.health > 0: # Если враг жив, движение вниз
            if self.rect.y <= HEIGHT:
                move(self, 'down')
            else:
                self.kill()

def cut_sheet(self, sheet, columns, rows):
    """Функция для деления изображения на фреймы и их последующего
анимирования"""
    self.rect = pygame.Rect(0, 0, sheet.get_width() // columns,
                             sheet.get_height() // rows)
    for j in range(rows):
        for i in range(columns):
            frame_location = (self.rect.w * i, self.rect.h * j)
            self.frames.append(sheet.subsurface(pygame.Rect(
                frame_location, self.rect.size)))

class Meteorite(Enemy):
    """Класс для метеоритов"""
    def __init__(self):
        super().__init__()
        self.health = 10 # Здоровье
        self.score = self.health # Очки равны здоровью

```

```

        self.speed = 40 # Скорость движения
        self.image =
pygame.transform.scale(load_image("Picture/Meteorite_1.png"), (80, 80)) #
Изображение
        self.rect = self.image.get_rect() # Занимаемый прямоугольник
        self.mask = pygame.mask.from_surface(self.image) # Маска для
коллизий

class MeteoriteBoss(Enemy):
    """Класс для метеоритов-боссов"""
    def __init__(self):
        super().__init__()
        self.health = 100 # Здоровье
        self.score = self.health # Очки
        self.speed = 15 # Скорость
        self.image =
pygame.transform.scale(load_image("Picture/Meteorite_2.png"), (150, 150)) #
Изображение
        self.rect = self.image.get_rect() # Занимаемый прямоугольник
        self.mask = pygame.mask.from_surface(self.image) # Маска для
коллизий

class Asteroid(Enemy):
    """Класс для астероидов"""
    def __init__(self):
        super().__init__()
        self.health = 30 # Здоровье
        self.score = self.health # Очки
        self.speed = 20 # Скорость
        self.image = load_image("Picture/Asteroid_1.png") # Изображение
        self.rect = self.image.get_rect() # Занимаемый прямоугольник
        self.mask = pygame.mask.from_surface(self.image) # Маска для
коллизий

class AsteroidBoss(Enemy):
    """Класс для астероидов-боссов"""
    def __init__(self):
        super().__init__()
        self.health = 150 # Здоровье
        self.score = self.health # Очки
        self.speed = 15 # Скорость
        self.image =
pygame.transform.scale(load_image("Picture/Asteroid_2.png"), (150, 150)) #
Изображение
        self.rect = self.image.get_rect() # Занимаемый прямоугольник
        self.mask = pygame.mask.from_surface(self.image) # Маска для
коллизий

class Comet(Enemy):
    """Класс для комет"""
    def __init__(self):
        super().__init__()
        self.health = 20 # Здоровье
        self.score = self.health # Очки
        self.speed = 25 # Скорость
        self.image = load_image("Picture/Comet.png") # Изображение
        self.rect = self.image.get_rect() # Занимаемый прямоугольник
        self.mask = pygame.mask.from_surface(self.image) # Маска для
коллизий

```

```

class Satellite(Enemy):
    """Класс для спутников"""
    def __init__(self):
        super().__init__()
        self.health = 10 # Здоровье
        self.score = -60 # Очки
        self.speed = 15 # Скорость
        self.image =
pygame.transform.scale(load_image("Picture/Satellite.png"), (80, 80)) #
Изображение
        self.rect = self.image.get_rect() # Занимаемый прямоугольник
        self.mask = pygame.mask.from_surface(self.image) # Маска для
коллизий

class Shot(pygame.sprite.Sprite):
    """Класс для выстрелов"""
    def __init__(self, x, y):
        super().__init__(all_sprites) # Добавление в группу всех спрайтов
        self.frames = [] # Кадры анимации
        self.cut_sheet(load_image("Picture/shot.png")) # Нарезка спрайт-
листа
        self.cur_frame = 0 # Текущий кадр
        self.k = 0 # Счётчик для анимации
        self.speed = 70 # Скорость полёта
        self.image = self.frames[self.cur_frame] # Изображение
        self.rect.x = x + 57 # Позиция относительно корабля
        self.rect.y = y - 30
        self.mask = pygame.mask.from_surface(self.image) # Маска для
коллизий

    def cut_sheet(self, sheet):
        """Функция для деления изображения на фреймы и их последующего
анимирования"""
        self.rect = pygame.Rect(0, 0, sheet.get_width() // 5,
                                sheet.get_height())

        for i in range(5):
            frame_location = (self.rect.w * i, 0)
            self.frames.append(sheet.subsurface(pygame.Rect(
                frame_location, self.rect.size)))

    def update(self):
        """Функция для обновления состояния выстрелов"""
        if self.rect.y < 0:
            SHOTS.remove(self)
            self.kill()
        self.k += 1
        if self.k % 5 == 0:
            self.cur_frame = (self.cur_frame + 1) % len(self.frames)
            self.image = self.frames[self.cur_frame]
            move(self, 'up')

def move(obj, rule):
    """Функция для движения всех объектов"""
    if rule == 'left': # Движение влево
        obj.rect.x -= obj.speed * 3 / FPS
    if rule == 'right': # Движение вправо
        obj.rect.x += obj.speed * 3 / FPS
    if rule == 'down': # Движение вниз
        obj.rect.y += obj.speed * 3 / FPS
    if rule == 'up': # Движение вверх
        obj.rect.y -= obj.speed * 3 / FPS

```

```
# Настройка отображения окна
pygame.display.set_caption('Космическая оборона') # Заголовок
pygame_icon = pygame.image.load('data/Picture/icon.png') # Иконка
pygame.display.set_icon(pygame_icon)

# Настройка музыки
di = 'data/Music/'
pygame.mixer.music.load(di + 'menu_music.mp3')
pygame.mixer.music.play(-1)
s = pygame.mixer.Sound('data/Sound/shot.wav')

# Запуск игры со стартового экрана
start_screen()

# Завершение игры
terminate()
```


Работа программы



